

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

```
import pandas_datareader
import datetime
```

▼ Importing the stock prices

Tesla, Ford and GM (General Motors)

```
import yfinance as yf
```

```
start = datetime.datetime(2019, 1, 1)
end = datetime.datetime(2024, 8, 8)
tesla = yf.download("TSLA", start, end)
```

```
[*****100%*****] 1 of 1 completed
```

```
tesla.head()
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2019-01-02	20.406668	21.008667	19.920000	20.674667	20.674667	174879000
2019-01-03	20.466667	20.626667	19.825333	20.024000	20.024000	104478000
2019-01-04	20.400000	21.200001	20.181999	21.179333	21.179333	110911500
2019-01-07	21.448000	22.449333	21.183332	22.330667	22.330667	113268000
2019-01-08	22.797333	22.934000	21.801332	22.356667	22.356667	105127500

```
ford = yf.download("F", start, end)
gm = yf.download("GM", start, end)
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

```
ford.head()
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2019-01-02	7.53	8.02	7.48	7.90	6.017155	47494400
2019-01-03	7.97	7.99	7.78	7.78	5.925754	39172400
2019-01-04	7.91	8.12	7.85	8.08	6.154253	43039800
2019-01-07	8.11	8.37	8.03	8.29	6.314204	40729400
2019-01-08	8.42	8.54	8.35	8.37	6.375136	45644000

```
gm.head()
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2019-01-02	32.849998	33.810001	32.410000	33.639999	31.238291	7662300
2019-01-03	33.340000	33.680000	32.200001	32.250000	29.947521	11618300
2019-01-04	32.799999	33.570000	32.430000	33.330002	30.950422	10784000
2019-01-07	33.709999	34.970001	33.480000	34.360001	31.906881	10722300
2019-01-08	35.000000	35.680000	34.490002	34.810001	32.324757	10532900

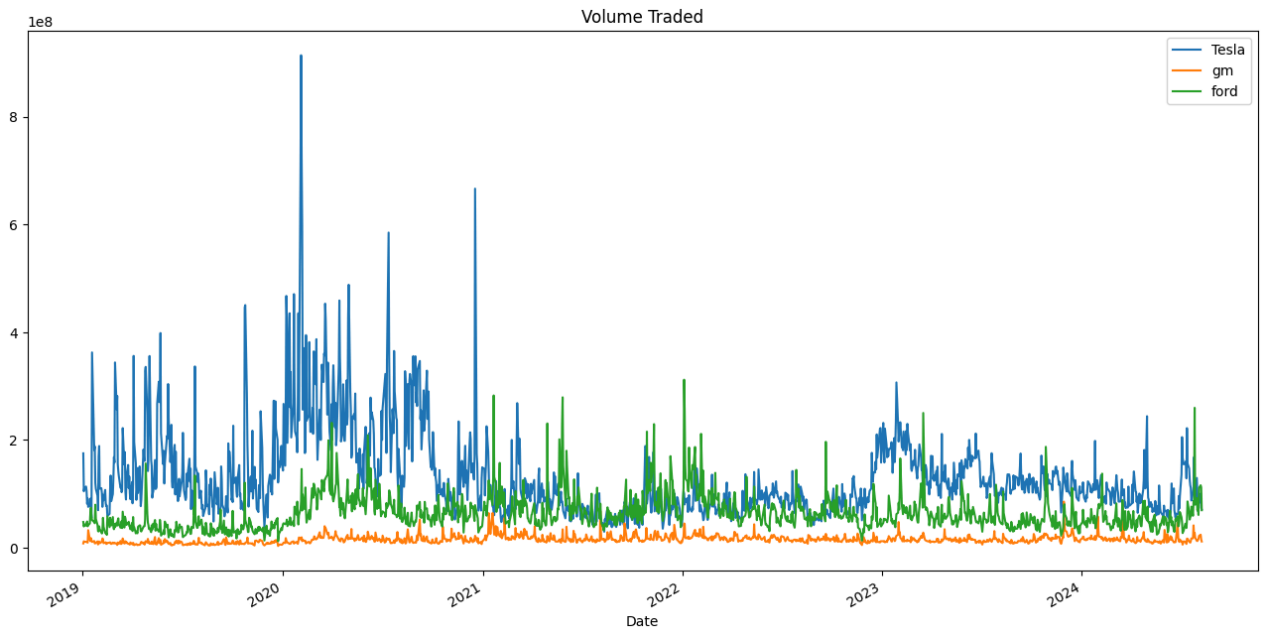
▼ Visualizing the Data

```
tesla['Open'].plot(label='Tesla',figsize=(16,8),title='Open Price')
gm['Open'].plot(label='GM')
ford['Open'].plot(label='Ford')
plt.legend()
plt.show()
```



Volume of stock traded each day

```
tesla['Volume'].plot(label='Tesla',figsize=(16,8),title='Volume Traded')
gm['Volume'].plot(label='gm')
ford['Volume'].plot(label='ford')
plt.legend()
plt.show()
```



Date of this maximum trading volume for Tesla, Ford and GM?

```
tesla['Volume'].idxmax()
```

```
Timestamp('2020-02-04 00:00:00')
```

```
ford['Volume'].idxmax()
```

```
Timestamp('2022-01-04 00:00:00')
```

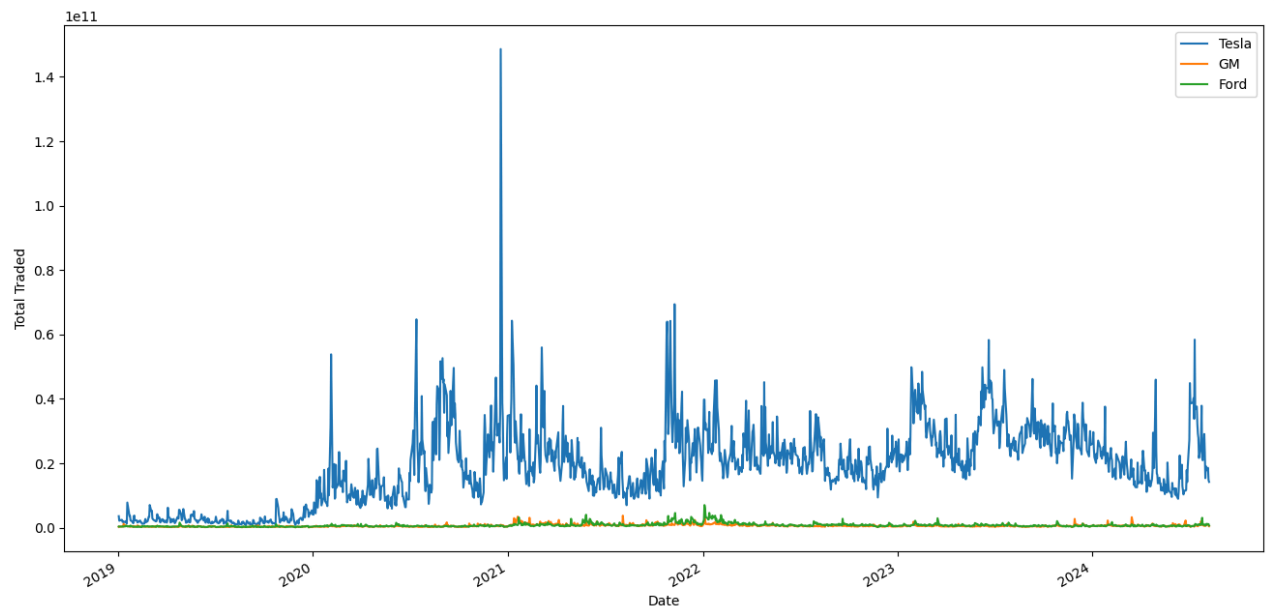
```
gm['Volume'].idxmax()
```

```
Timestamp('2023-11-29 00:00:00')
```

Total Traded

```
tesla['Total Traded'] = tesla['Open']*tesla['Volume']
ford['Total Traded'] = ford['Open']*ford['Volume']
gm['Total Traded'] = gm['Open']*gm['Volume']
```

```
tesla['Total Traded'].plot(label='Tesla',figsize=(16,8))
gm['Total Traded'].plot(label='GM')
ford['Total Traded'].plot(label='Ford')
plt.legend()
plt.ylabel('Total Traded')
plt.show()
```

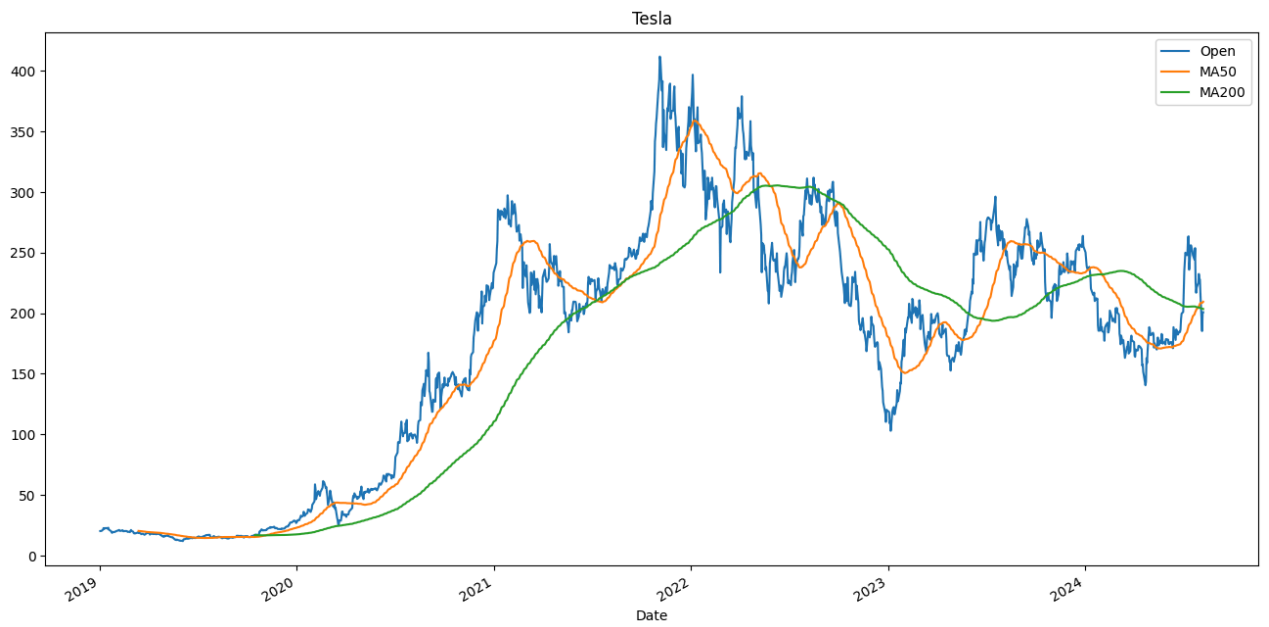


```
tesla['Total Traded'].idxmax()
```

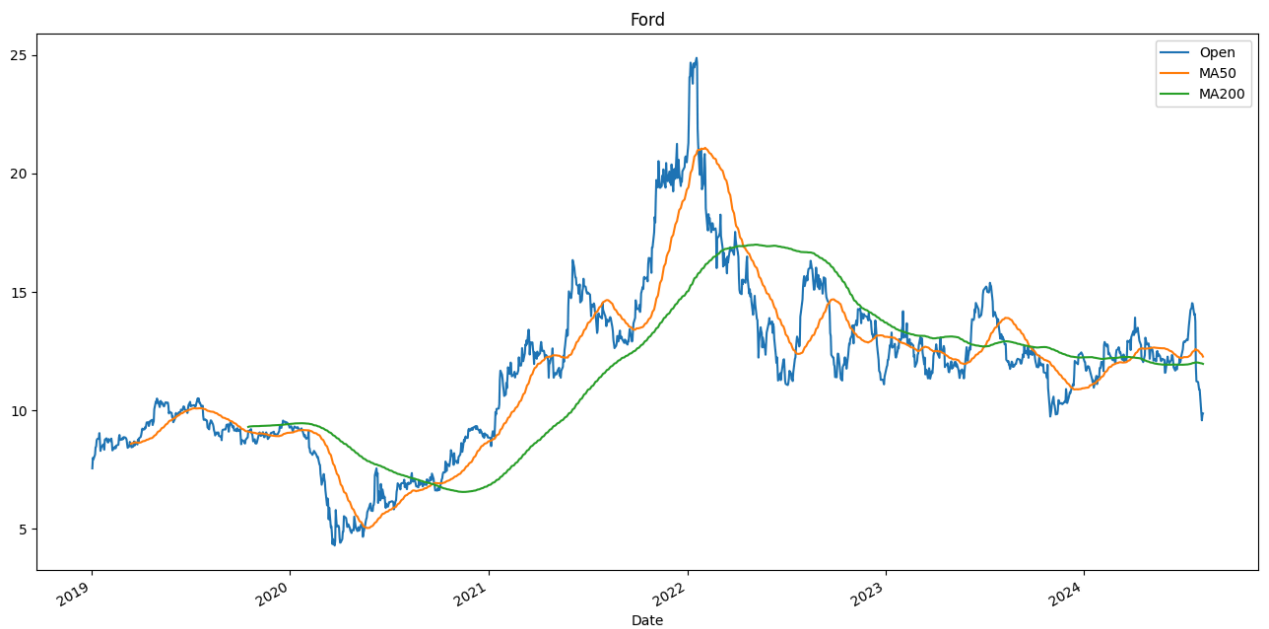
```
Timestamp('2020-12-18 00:00:00')
```

Moving Averages

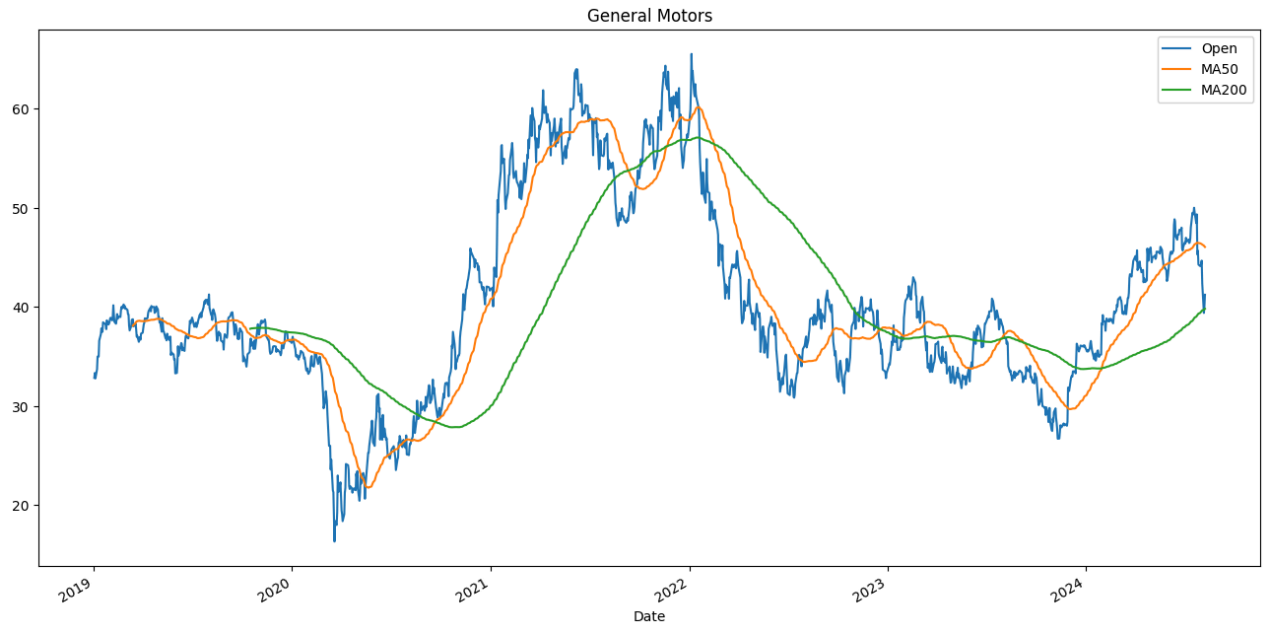
```
tesla['MA50'] = tesla['Open'].rolling(50).mean()
tesla['MA200'] = tesla['Open'].rolling(200).mean()
tesla[['Open', 'MA50', 'MA200']].plot(label='tesla', figsize=(16,8))
plt.title('Tesla')
plt.show()
```



```
ford['MA50'] = ford['Open'].rolling(50).mean()
ford['MA200'] = ford['Open'].rolling(200).mean()
ford[['Open', 'MA50', 'MA200']].plot(label='tesla', figsize=(16,8))
plt.title('Ford')
plt.show()
```



```
gm['MA50'] = gm['Open'].rolling(50).mean()
gm['MA200'] = gm['Open'].rolling(200).mean()
gm[['Open', 'MA50', 'MA200']].plot(label='tesla', figsize=(16,8))
plt.title('General Motors')
plt.show()
```



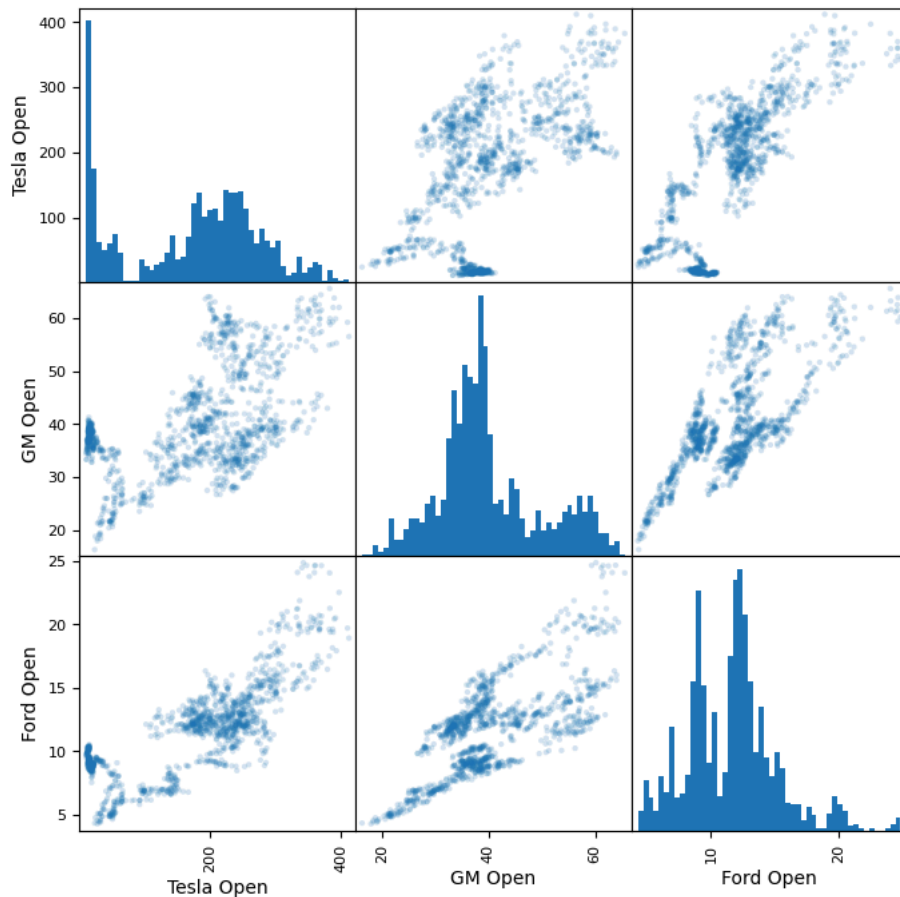
Relationship among the three stocks

```
from pandas.plotting import scatter_matrix
```

```
car_comp = pd.concat([tesla['Open'], gm['Open'], ford['Open']], axis=1)
```

```
car_comp.columns = ['Tesla Open', 'GM Open', 'Ford Open']
```

```
scatter_matrix(car_comp, figsize=(8,8), alpha=0.2, hist_kwds={'bins':50});
```



Candlestick Chart

```
from mplfinance.original_flavor import candlestick_ohlc
from matplotlib.dates import DateFormatter, date2num, WeekdayLocator, DayLocator, MONDAY

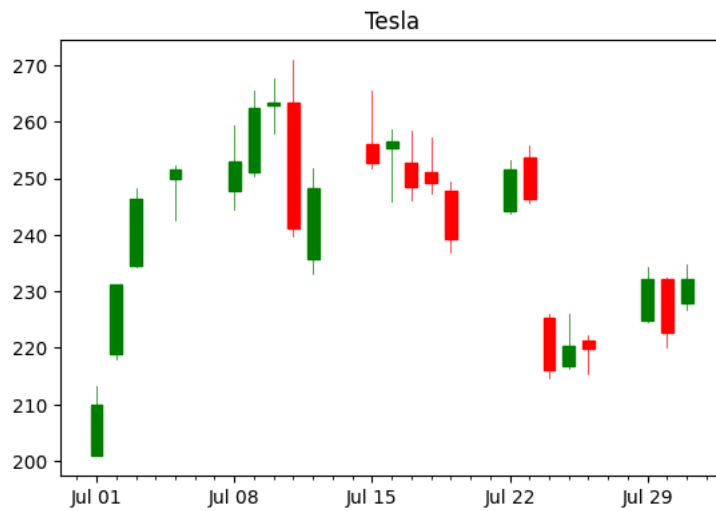
# Reset the index to get a column of January Dates
tesla_reset = tesla.loc['2024-07':'2024-07'].reset_index()

# Create a new column of numerical "date" values for matplotlib to use
tesla_reset['date_ax'] = tesla_reset['Date'].apply(lambda date: date2num(date))
tesla_values = [tuple(vals) for vals in tesla_reset[['date_ax', 'Open', 'High', 'Low', 'Close']].values]

mondays = WeekdayLocator(MONDAY)
alldays = DayLocator()
weekFormatter = DateFormatter('%b %d')
dayFormatter = DateFormatter('%d')

#Plot it
fig, ax = plt.subplots()
fig.subplots_adjust(bottom=0.2)
ax.xaxis.set_major_locator(mondays)
ax.xaxis.set_minor_locator(alldays)
ax.xaxis.set_major_formatter(weekFormatter)

candlestick_ohlc(ax, tesla_values, width=0.6, colorup='g',colordown='r')
plt.title('Tesla')
plt.show()
```



```
from mplfinance.original_flavor import candlestick_ohlc
from matplotlib.dates import DateFormatter, date2num, WeekdayLocator, DayLocator, MONDAY

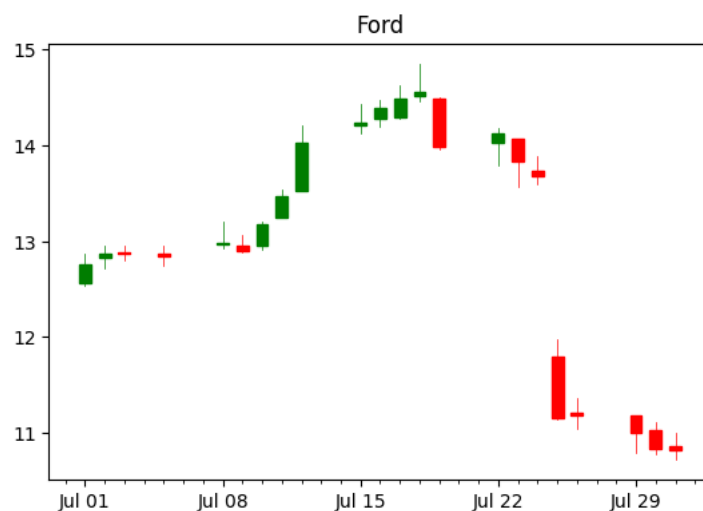
# Reset the index to get a column of January Dates
ford_reset = ford.loc['2024-07':'2024-07'].reset_index()

# Create a new column of numerical "date" values for matplotlib to use
ford_reset['date_ax'] = ford_reset['Date'].apply(lambda date: date2num(date))
ford_values = [tuple(vals) for vals in ford_reset[['date_ax', 'Open', 'High', 'Low', 'Close']].values]

mondays = WeekdayLocator(MONDAY)
alldays = DayLocator()
weekFormatter = DateFormatter('%b %d')
dayFormatter = DateFormatter('%d')

#Plot it
fig, ax = plt.subplots()
fig.subplots_adjust(bottom=0.2)
ax.xaxis.set_major_locator(mondays)
ax.xaxis.set_minor_locator(alldays)
ax.xaxis.set_major_formatter(weekFormatter)

candlestick_ohlc(ax, ford_values, width=0.6, colorup='g', colordown='r')
plt.title('Ford')
plt.show()
```



```
from mplfinance.original_flavor import candlestick_ohlc
from matplotlib.dates import DateFormatter, date2num, WeekdayLocator, DayLocator, MONDAY

# Reset the index to get a column of January Dates
gm_reset = gm.loc['2024-07':'2024-07'].reset_index()

# Create a new column of numerical "date" values for matplotlib to use
gm_reset['date_ax'] = gm_reset['Date'].apply(lambda date: date2num(date))
gm_values = [tuple(vals) for vals in gm_reset[['date_ax', 'Open', 'High', 'Low', 'Close']].values]

mondays = WeekdayLocator(MONDAY)
```

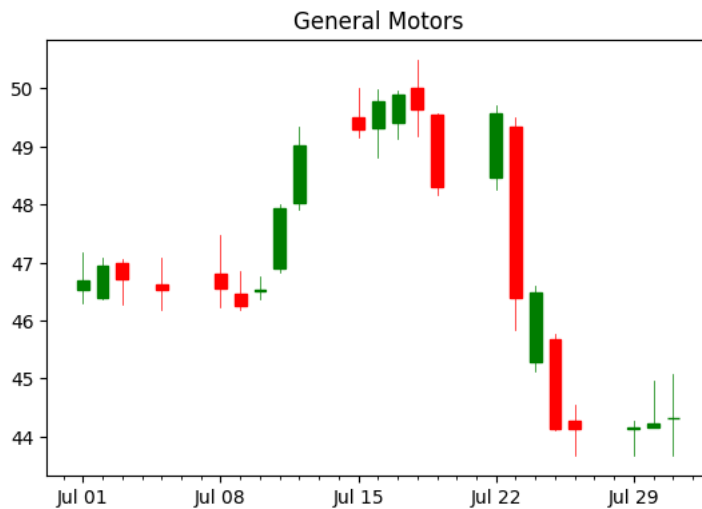
```

alldays = DayLocator()
weekFormatter = DateFormatter('%b %d')
dayFormatter = DateFormatter('%d')

#Plot it
fig, ax = plt.subplots()
fig.subplots_adjust(bottom=0.2)
ax.xaxis.set_major_locator(mondays)
ax.xaxis.set_minor_locator(alldays)
ax.xaxis.set_major_formatter(weekFormatter)

candlestick_ohlc(ax, gm_values, width=0.6, colorup='g',colordown='r')
plt.title('General Motors')
plt.show()

```



Basic Financial Analysis

Daily Percentage Change

```

tesla['returns'] = tesla['Close'].pct_change(1)
ford['returns'] = ford['Close'].pct_change(1)
gm['returns'] = gm['Close'].pct_change(1)

```

```
tesla.head()
```

	Open	High	Low	Close	Adj Close	Volume	Total Traded	MA50	MA200	returns
Date										
2019-01-02	20.406668	21.008667	19.920000	20.674667	20.674667	174879000	3.568698e+09	NaN	NaN	NaN
2019-01-03	20.466667	20.626667	19.825333	20.024000	20.024000	104478000	2.138316e+09	NaN	NaN	-0.031472
2019-01-04	20.400000	21.200001	20.181999	21.179333	21.179333	110911500	2.262595e+09	NaN	NaN	0.057697
2019-01-07	21.448000	22.449333	21.183332	22.330667	22.330667	113268000	2.429372e+09	NaN	NaN	0.054361
2019-01-08	22.797333	22.934000	21.801332	22.356667	22.356667	105127500	2.396627e+09	NaN	NaN	0.001164

```
ford.head()
```

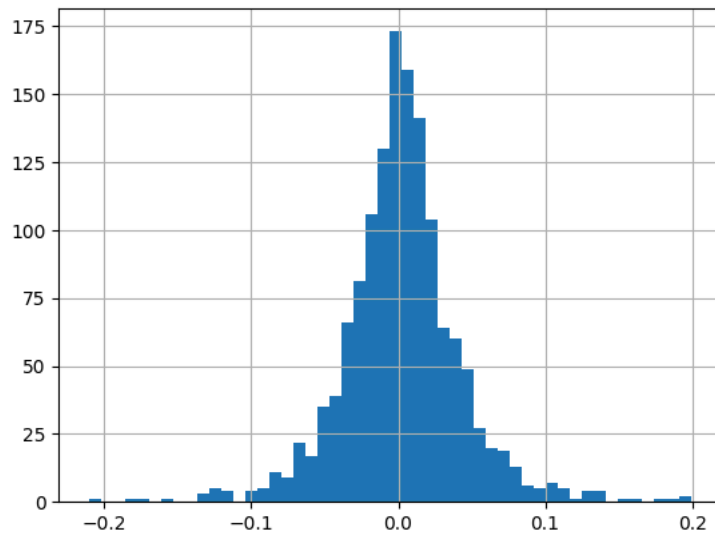
	Open	High	Low	Close	Adj Close	Volume	Total Traded	MA50	MA200	returns
Date										
2019-01-02	7.53	8.02	7.48	7.90	6.017155	47494400	3.576328e+08	NaN	NaN	NaN
2019-01-03	7.97	7.99	7.78	7.78	5.925754	39172400	3.122040e+08	NaN	NaN	-0.01519
2019-01-04	7.91	8.12	7.85	8.08	6.154253	43039800	3.404448e+08	NaN	NaN	0.03856
2019-01-07	8.11	8.37	8.03	8.29	6.314204	40729400	3.303154e+08	NaN	NaN	0.02599
2019-01-08	8.42	8.54	8.35	8.37	6.375136	45644000	3.843225e+08	NaN	NaN	0.00965

```
gm.head()
```

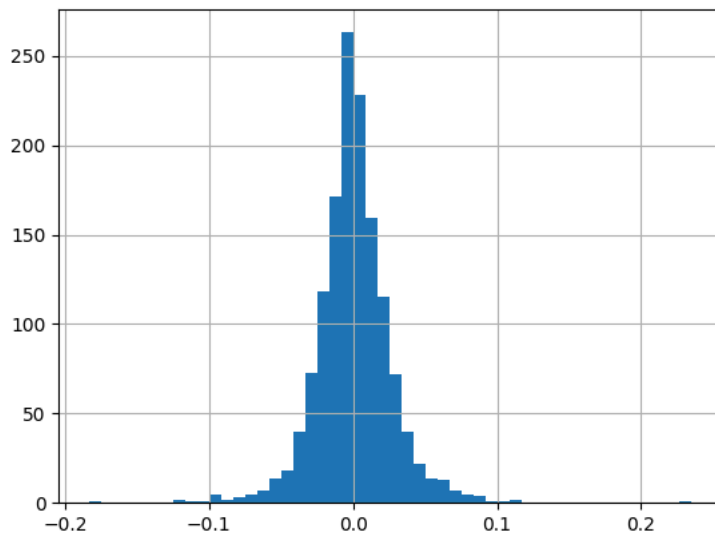
	Open	High	Low	Close	Adj Close	Volume	Total Traded	MA50	MA200	returns
Date										
2019-01-02	32.849998	33.810001	32.410000	33.639999	31.238291	7662300	2.517065e+08	NaN	NaN	NaN
2019-01-03	33.340000	33.680000	32.200001	32.250000	29.947521	11618300	3.873541e+08	NaN	NaN	-0.041320
2019-01-04	32.799999	33.570000	32.430000	33.330002	30.950422	10784000	3.537152e+08	NaN	NaN	0.033488
2019-01-07	33.709999	34.970001	33.480000	34.360001	31.906881	10722300	3.614487e+08	NaN	NaN	0.030903
2019-01-08	35.000000	35.680000	34.490002	34.810001	32.324757	10532900	3.686515e+08	NaN	NaN	0.013097

Checking volatility

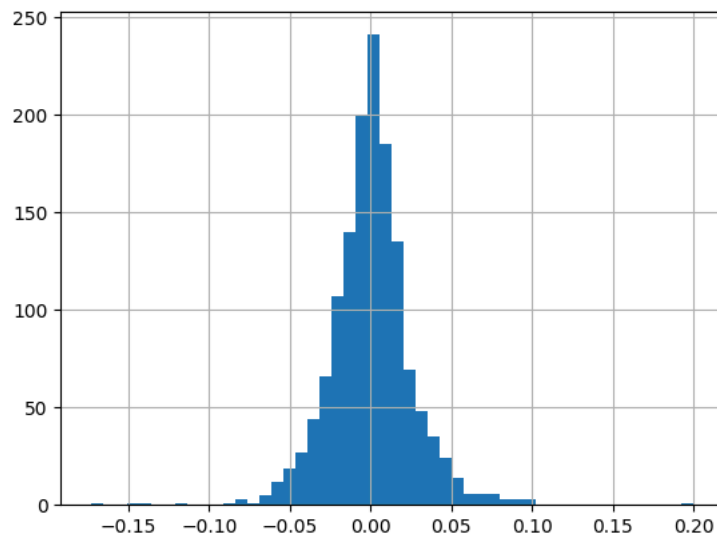
```
tesla['returns'].hist(bins=50)
plt.show()
```



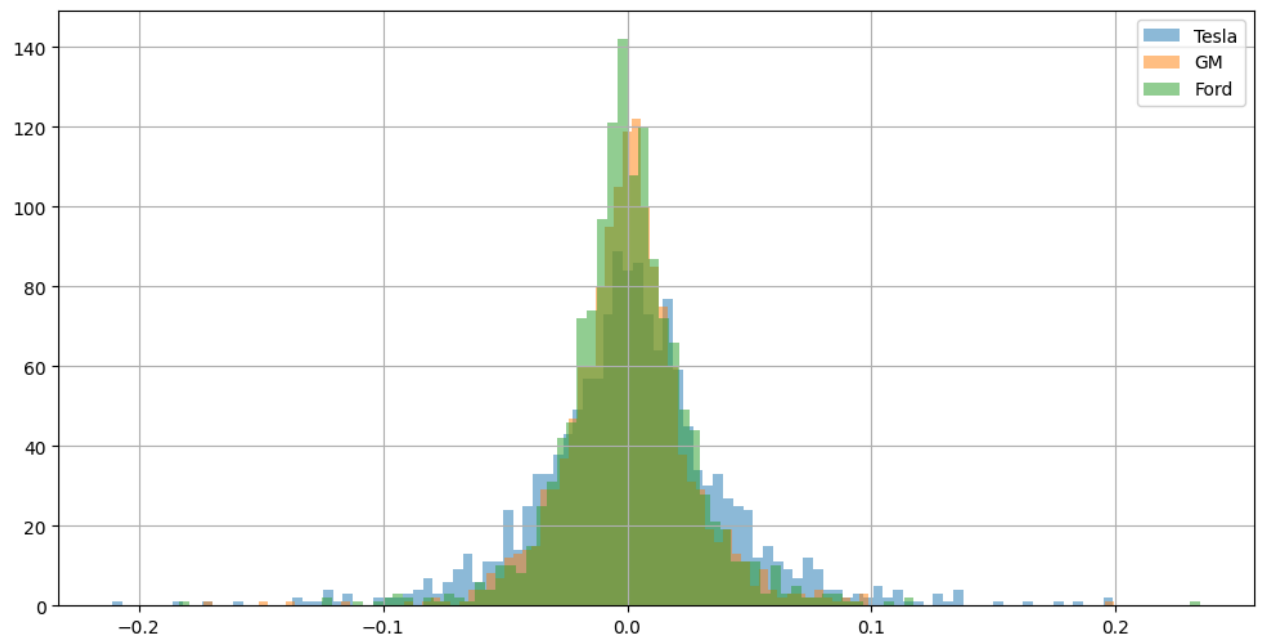
```
ford['returns'].hist(bins=50)
plt.show()
```



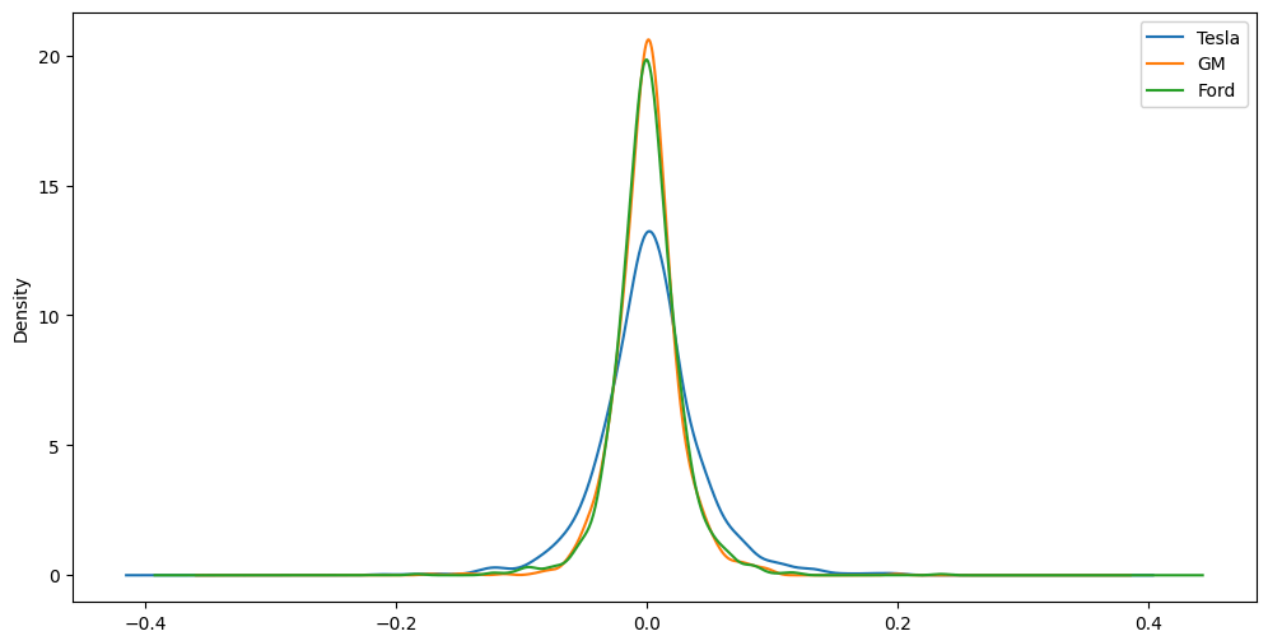
```
gm['returns'].hist(bins=50)
plt.show()
```



```
tesla['returns'].hist(bins=100,label='Tesla',figsize=(12,6),alpha=0.5)
gm['returns'].hist(bins=100,label='GM',alpha=0.5)
ford['returns'].hist(bins=100,label='Ford',alpha=0.5)
plt.legend()
plt.show()
```

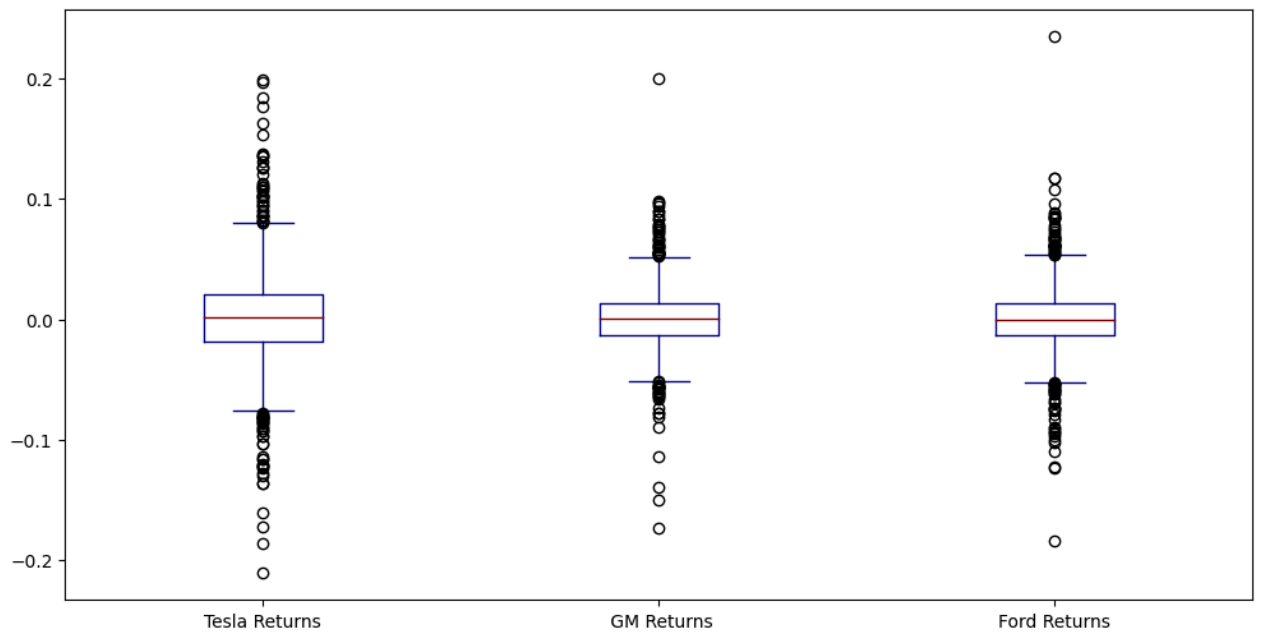


```
tesla['returns'].plot(kind='kde',label='Tesla',figsize=(12,6))
gm['returns'].plot(kind='kde',label='GM')
ford['returns'].plot(kind='kde',label='Ford')
plt.legend()
plt.show()
```



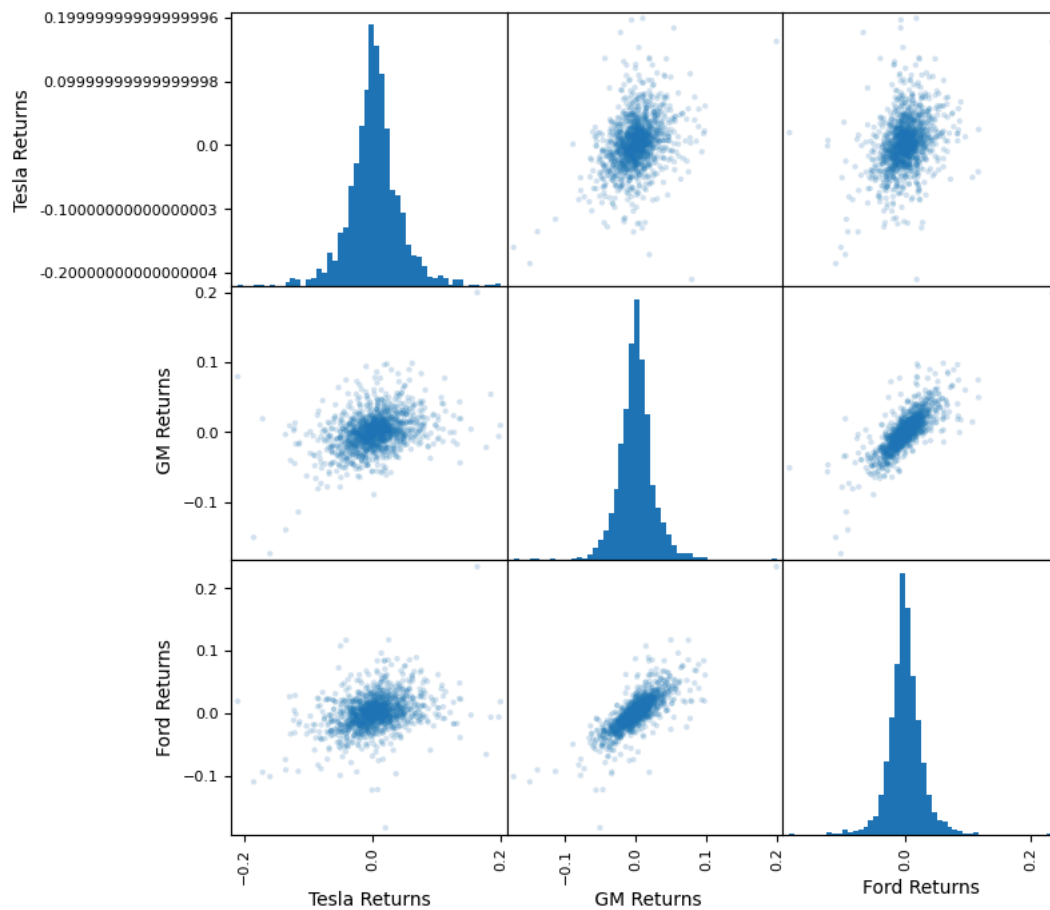
```
box_df = pd.concat([tesla['returns'],gm['returns'],ford['returns']],axis=1)
box_df.columns = ['Tesla Returns',' GM Returns','Ford Returns']
box_df.plot(kind='box',figsize=(12,6),colormap='jet')
```

<Axes: >



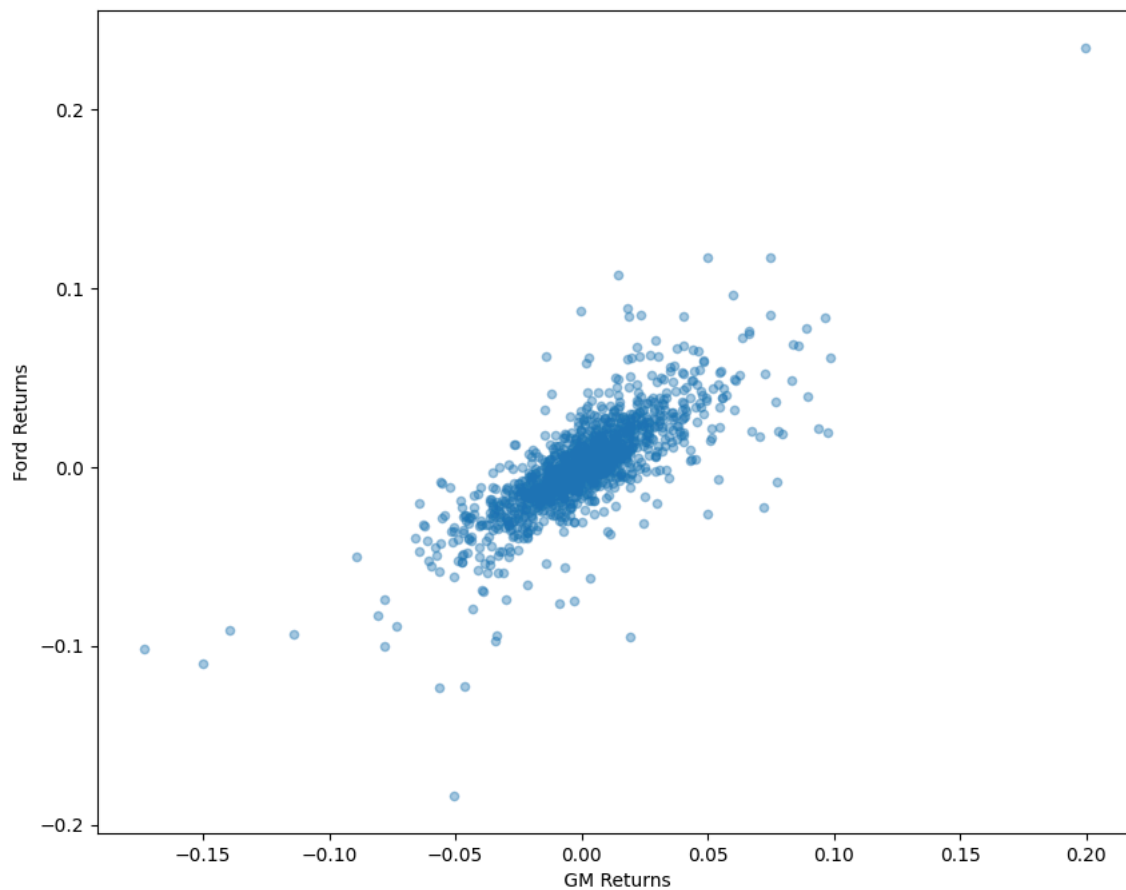
Comparing Daily Returns between Stocks

```
scatter_matrix(box_df,figsize=(8,8),alpha=0.2,hist_kws={'bins':50});
```



It looks like Ford and GM do have some sort of possible relationship, let's plot just these two against each other in scatter plot to view this more closely!

```
box_df.plot(kind='scatter',x=' GM Returns',y='Ford Returns',alpha=0.4,figsize=(10,8))
plt.show()
```



Cumulative Daily Returns

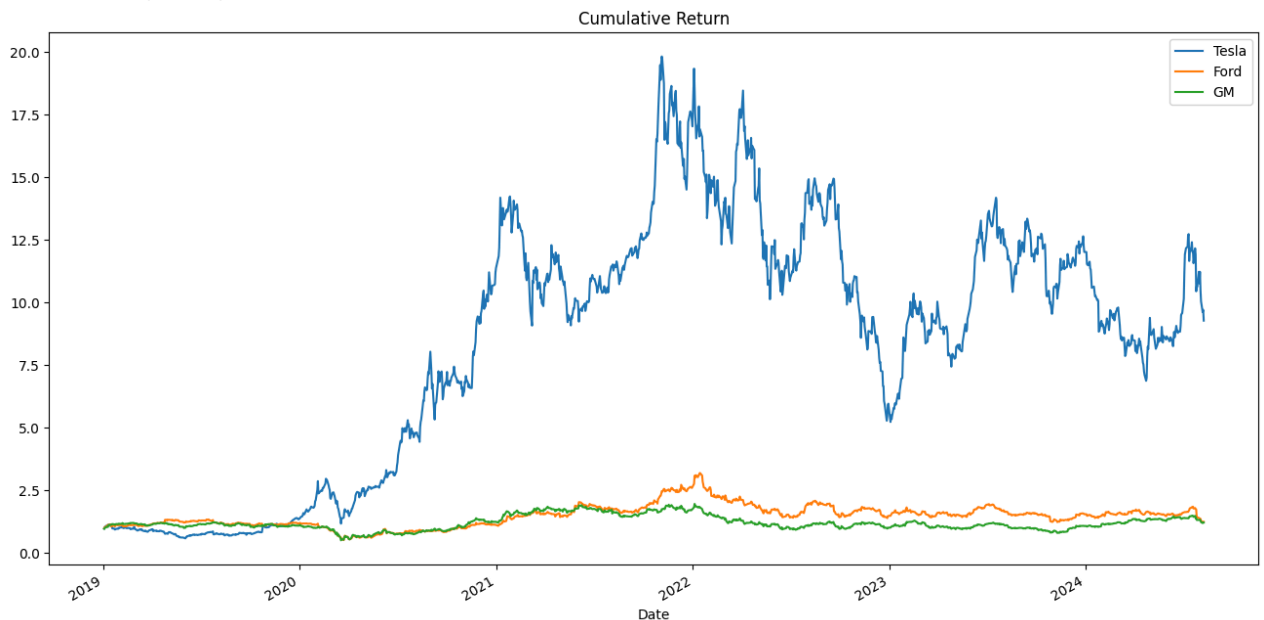
```
tesla['Cumulative Return'] = (1 + tesla['returns']).cumprod()
ford['Cumulative Return'] = (1 + ford['returns']).cumprod()
gm['Cumulative Return'] = (1 + gm['returns']).cumprod()
```

```
gm.head()
```

	Open	High	Low	Close	Adj Close	Volume	Total Traded	MA50	MA200	returns	Cumulative Return
Date											
2019-01-02	32.849998	33.810001	32.410000	33.639999	31.238291	7662300	2.517065e+08	NaN	NaN	NaN	NaN
2019-01-03	33.340000	33.680000	32.200001	32.250000	29.947521	11618300	3.873541e+08	NaN	NaN	-0.041320	0.958680
2019-01-04	32.799999	33.570000	32.430000	33.330002	30.950422	10784000	3.537152e+08	NaN	NaN	0.033488	0.990785

```
tesla['Cumulative Return'].plot(label='Tesla',figsize=(16,8),title='Cumulative Return')
ford['Cumulative Return'].plot(label='Ford')
gm['Cumulative Return'].plot(label='GM')
plt.legend()
```

<matplotlib.legend.Legend at 0x1e29e71f950>



Portfolio Creation

Let's pretend we had the following allocations for our total portfolio:

- 30% in Tesla
- 20% in Ford
- 50% in General Motors

Let's have these values be reflected by multiplying our Norme Return by out Allocations

```
for stock_df in (tesla,ford,gm):
    stock_df['Normed Return'] = stock_df['Adj Close']/stock_df.iloc[0]['Adj Close']
```

```
for stock_df,allo in zip([tesla, ford, gm],[.3,.2,.5]):
    stock_df['Allocation'] = stock_df['Normed Return']*allo
```

```
tesla.head()
```

	Open	High	Low	Close	Adj Close	Volume	Total Traded	MA50	MA200	returns	Cumulative Return	Norm Retu
Date												
2019-01-02	20.406668	21.008667	19.920000	20.674667	20.674667	174879000	3.568698e+09	NaN	NaN	NaN	NaN	1.0000
2019-01-03	20.466667	20.626667	19.825333	20.024000	20.024000	104478000	2.138316e+09	NaN	NaN	-0.031472	0.968528	0.9685
2019-01-04	20.400000	21.200001	20.181999	21.179333	21.179333	110911500	2.262595e+09	NaN	NaN	0.057697	1.024410	1.0244

Let's pretend we invested a million dollars in this portfolio

```
for stock_df in [tesla, ford, gm]:
    stock_df['Position Values'] = stock_df['Allocation']*1000000
```

Total Portfolio Value

```
portfolio_val = pd.concat([tesla['Position Values'],ford['Position Values'],gm['Position Values']],axis=1)
```

```
portfolio_val.columns = ['Tesla Pos','Ford Pos','GM Pos']
```

```
portfolio_val.head()
```

	Tesla Pos	Ford Pos	GM Pos
Date			
2019-01-02	300000.000000	200000.000000	500000.000000
2019-01-03	290558.486200	196961.982704	479339.945552
2019-01-04	307322.952762	204556.914999	495392.377552
2019-01-07	324029.409160	209873.421493	510701.458512
2019-01-08	324406.668955	211898.687111	517389.969301

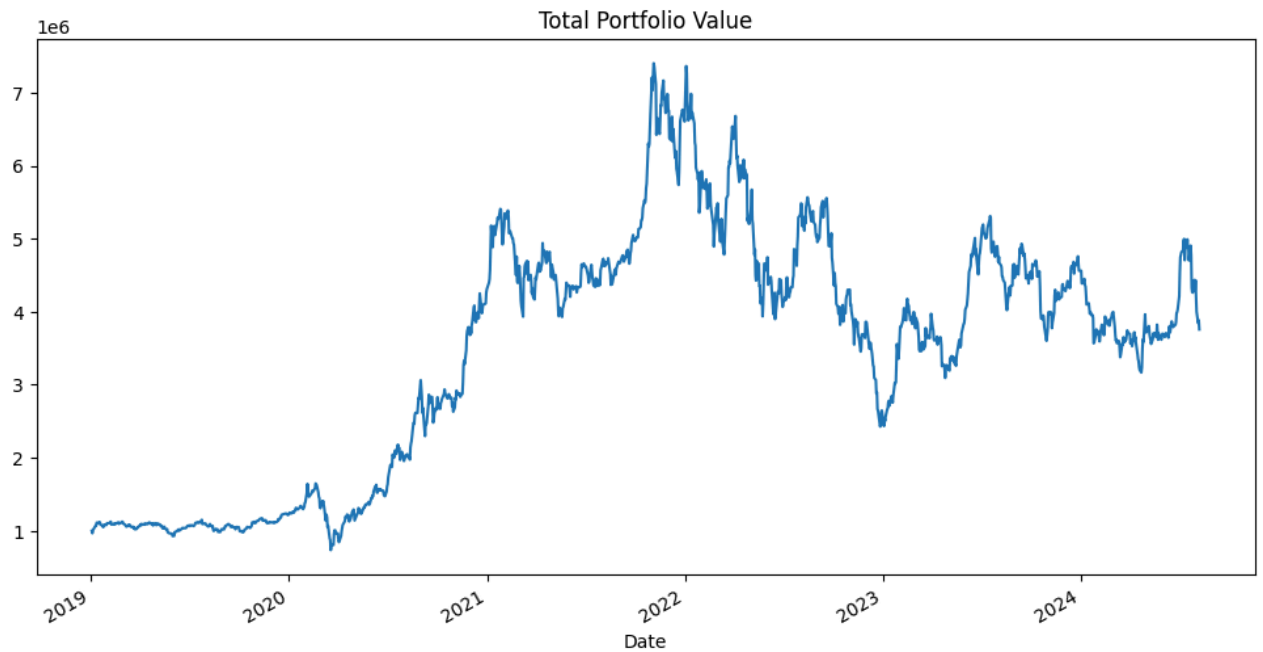
```
portfolio_val['Total Pos'] = portfolio_val.sum(axis=1)
```

```
portfolio_val.head()
```

	Tesla Pos	Ford Pos	GM Pos	Total Pos
Date				
2019-01-02	300000.000000	200000.000000	500000.000000	1.000000e+06
2019-01-03	290558.486200	196961.982704	479339.945552	9.668604e+05
2019-01-04	307322.952762	204556.914999	495392.377552	1.007272e+06
2019-01-07	324029.409160	209873.421493	510701.458512	1.044604e+06
2019-01-08	324406.668955	211898.687111	517389.969301	1.053695e+06

```
import matplotlib.pyplot as plt
%matplotlib inline
```

```
portfolio_val['Total Pos'].plot(figsize=(12,6))
plt.title('Total Portfolio Value')
plt.show()
```



```
portfolio_val.drop('Total Pos',axis=1).plot(kind='line',figsize=(12,6))  
plt.show()
```



```
portfolio_val.tail()
```

	Tesla Pos	Ford Pos	GM Pos	Total Pos
Date				
2024-08-01	3.146750e+06	349867.785497	694180.088596	4.190797e+06
2024-08-02	3.013398e+06	328266.959815	658966.882183	4.000632e+06
2024-08-05	2.885851e+06	317793.865830	639439.606918	3.843084e+06
2024-08-06	2.911389e+06	320084.844243	650003.562736	3.881477e+06
2024-08-07	2.782536e+06	324738.217821	650643.817285	3.757918e+06

Portfolio Statistics

Daily Returns

```
portfolio_val['Daily Return'] = portfolio_val['Total Pos'].pct_change(1)
portfolio_val.head()
```

	Tesla Pos	Ford Pos	GM Pos	Total Pos	Daily Return
Date					
2019-01-02	300000.000000	200000.000000	500000.000000	1.000000e+06	NaN
2019-01-03	290558.486200	196961.982704	479339.945552	9.668604e+05	-0.033140
2019-01-04	307322.952762	204556.914999	495392.377552	1.007272e+06	0.041797
2019-01-07	324029.409160	209873.421493	510701.458512	1.044604e+06	0.037063
2019-01-08	324406.668955	211898.687111	517389.969301	1.053695e+06	0.008703

Cumulative Return

```
cum_ret = 100 * (portfolio_val['Total Pos'][-1]/portfolio_val['Total Pos'][0]-1)
print('Our return {} was percent!'.format(cum_ret))
```

Our return 275.7917654895466 was percent!

```
portfolio_val['Total Pos'][-1]
```

3757917.6548954663

Avg Daily Return

```
portfolio_val['Daily Return'][2:].mean(numeric_only=True)
```

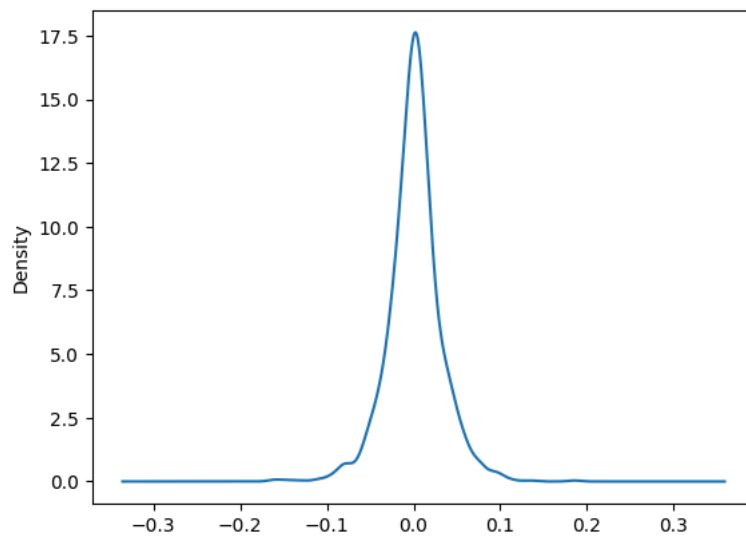
0.0014481232538196573

Std Daily Return

```
portfolio_val['Daily Return'].std()
```

0.031039258048563518

```
portfolio_val['Daily Return'].plot(kind='kde')
plt.show()
```



Sharpe Ratio

```
SR = portfolio_val['Daily Return'].mean()/portfolio_val['Daily Return'].std()
SR
```

0.045863149096651415

```
ASR = (252**0.5)*SR
ASR
```

```
0.7280549211120977
```

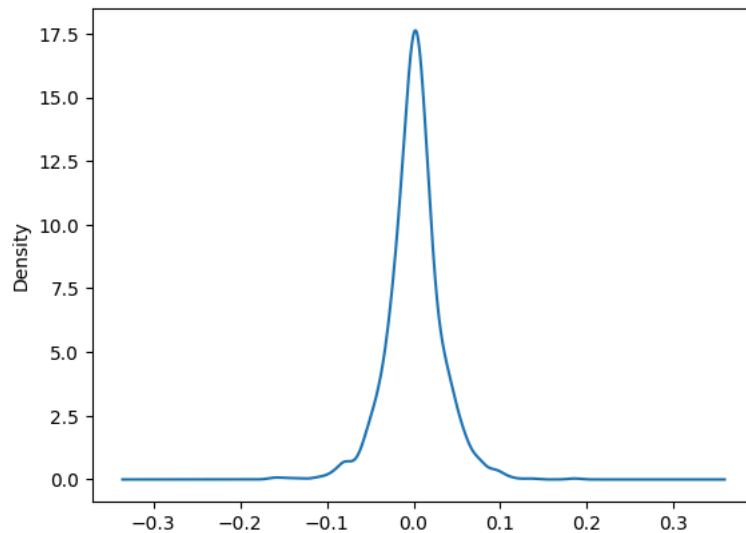
```
portfolio_val['Daily Return'].std()
```

```
0.031039258048563518
```

```
portfolio_val['Daily Return'].mean()
```

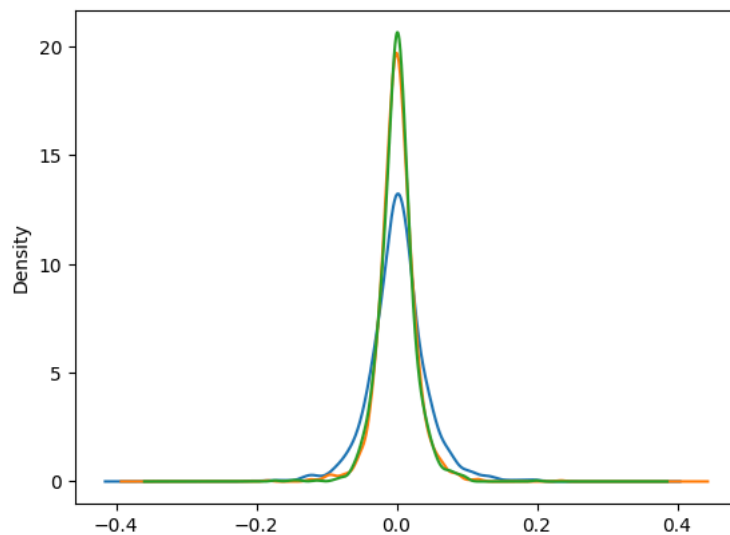
```
0.0014235581197307062
```

```
portfolio_val['Daily Return'].plot(kind='kde')  
plt.show()
```



```
tesla['Adj Close'].pct_change(1).plot(kind='kde')  
ford['Adj Close'].pct_change(1).plot(kind='kde')  
gm['Adj Close'].pct_change(1).plot(kind='kde')
```

<Axes: ylabel='Density'>



✓ Portfolio Optimization

✓ Monte Carlo Simulation for Optimization Search

We could randomly try to find the optimal portfolio balance using Monte Carlo simulation

```
stocks = pd.concat([tesla['Adj Close'],ford['Adj Close'],gm['Adj Close']],axis=1)  
stocks.columns = ['Tesla', 'Ford', 'General Motors']
```

```
stocks.head()
```

	Tesla	Ford	General Motors
Date			
2019-01-02	20.674667	6.017155	31.238291
2019-01-03	20.024000	5.925754	29.947521
2019-01-04	21.179333	6.154253	30.950422
2019-01-07	22.330667	6.314204	31.906881
2019-01-08	22.356667	6.375136	32.324757

```
mean_daily_ret = stocks.pct_change(1).mean()
mean_daily_ret
```

```
Tesla      0.002397
Ford       0.000708
General Motors 0.000520
dtype: float64
```

```
stocks.pct_change(1).corr()
```

	Tesla	Ford	General Motors
Tesla	1.000000	0.326015	0.354696
Ford	0.326015	1.000000	0.780001
General Motors	0.354696	0.780001	1.000000

```
import plotly.express as px
px.imshow(stocks.pct_change(1).corr(),text_auto=True)
```

Simulating Thousands of Possible Allocations

```
stock_normed = stocks/stocks.iloc[0]
stock_normed.plot(figsize=(10,6))
plt.show()
```



```
stock_daily_ret = stocks.pct_change(1)
stock_daily_ret.head()
```

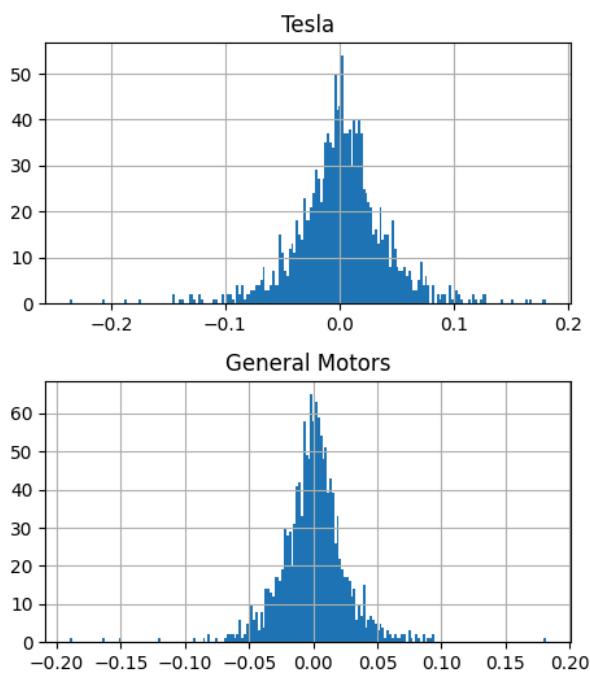
	Tesla	Ford	General Motors
Date			
2019-01-02	NaN	NaN	NaN
2019-01-03	-0.031472	-0.01519	-0.041320
2019-01-04	0.057697	0.03856	0.033489
2019-01-07	0.054361	0.02599	0.030903
2019-01-08	0.001164	0.00965	0.013097

Log Returns

```
log_ret = np.log(stocks/stocks.shift(1))
log_ret.head()
```

	Tesla	Ford	General Motors
Date			
2019-01-02	NaN	NaN	NaN
2019-01-03	-0.031978	-0.015307	-0.042198
2019-01-04	0.056094	0.037836	0.032940
2019-01-07	0.052935	0.025658	0.030435
2019-01-08	0.001164	0.009604	0.013012

```
log_ret.hist(bins=200,figsize=(12,6));
```



```
log_ret.describe().transpose()
```

	count	mean	std	min	25%	50%	75%	max
Tesla	1408.0	0.001582	0.040358	-0.236518	-0.018617	0.001910	0.020761	0.181445
Ford	1408.0	0.000344	0.026999	-0.202868	-0.012924	0.000000	0.013620	0.210596
General Motors	1408.0	0.000187	0.025824	-0.190227	-0.012709	0.000662	0.013181	0.181848

```
log_ret.mean() * 252
```

```
Tesla      0.398642
Ford       0.086751
General Motors 0.047134
dtype: float64
```

Pairwise covariance

```
log_ret.cov()
```

	Tesla	Ford	General Motors
Tesla	0.001629	0.000360	0.000376
Ford	0.000360	0.000729	0.000543
General Motors	0.000376	0.000543	0.000667

```
log_ret.cov()*252 # multiply by days
```

	Tesla	Ford	General Motors
Tesla	0.410450	0.090659	0.094773
Ford	0.090659	0.183697	0.136744
General Motors	0.094773	0.136744	0.168048

Single Run for Some Random Allocation

```
print('Stocks')
print(stocks.columns)
print('\n')

print('Creating Random Weights')
weights = np.array(np.random.random(3))
print(weights)
print('\n')

print('Rebalance to sum to 1.0')
weights = weights / np.sum(weights)
print(weights)
print('\n')

print('Expected Portfolio Return')
exp_ret = np.sum(log_ret.mean() * weights) * 252
print(exp_ret)
print('\n')

print('Expected Volatility')
exp_vol = np.sqrt(np.dot(weights.T, np.dot(log_ret.cov() * 252, weights)))
print(exp_vol)
print('\n')

SR = exp_ret/exp_vol
print('Sharpe Ratio')
print(SR)
```

```
Stocks
Index(['Tesla', 'Ford', 'General Motors'], dtype='object')
```

```
Creating Random Weights
[0.18993895 0.55422759 0.35213195]
```

```
Rebalance to sum to 1.0
[0.17325478 0.50554442 0.3212008 ]
```

```
Expected Portfolio Return
0.12806262356026976
```

```
Expected Volatility
0.383986011246186
```

```
Sharpe Ratio
0.333508564920519
```

Multiple Run

```
num_ports = 15000
```

```
all_weights = np.zeros((num_ports, len(stocks.columns)))
```

```

ret_arr = np.zeros(num_ports)
vol_arr = np.zeros(num_ports)
sharpe_arr = np.zeros(num_ports)

for ind in range(num_ports):

    # Create Random Weights
    weights = np.array(np.random.random(3))

    # Rebalance Weights
    weights = weights / np.sum(weights)

    # Save Weights
    all_weights[ind,:] = weights

    # Expected Return
    ret_arr[ind] = np.sum((log_ret.mean() * weights) * 252)

    # Expected Variance
    vol_arr[ind] = np.sqrt(np.dot(weights.T, np.dot(log_ret.cov() * 252, weights)))

    # Sharpe Ratio
    sharpe_arr[ind] = ret_arr[ind]/vol_arr[ind]

```

```
sharpe_arr.max()
```

```
0.6219882536876815
```

```
sharpe_arr.argmax()
```

```
1660
```

```
all_weights[1419,:]
```

```
array([0.01442151, 0.20142799, 0.7841505 ])
```

```
max_sr_ret = ret_arr[1419]
```

```
max_sr_vol = vol_arr[1419]
```

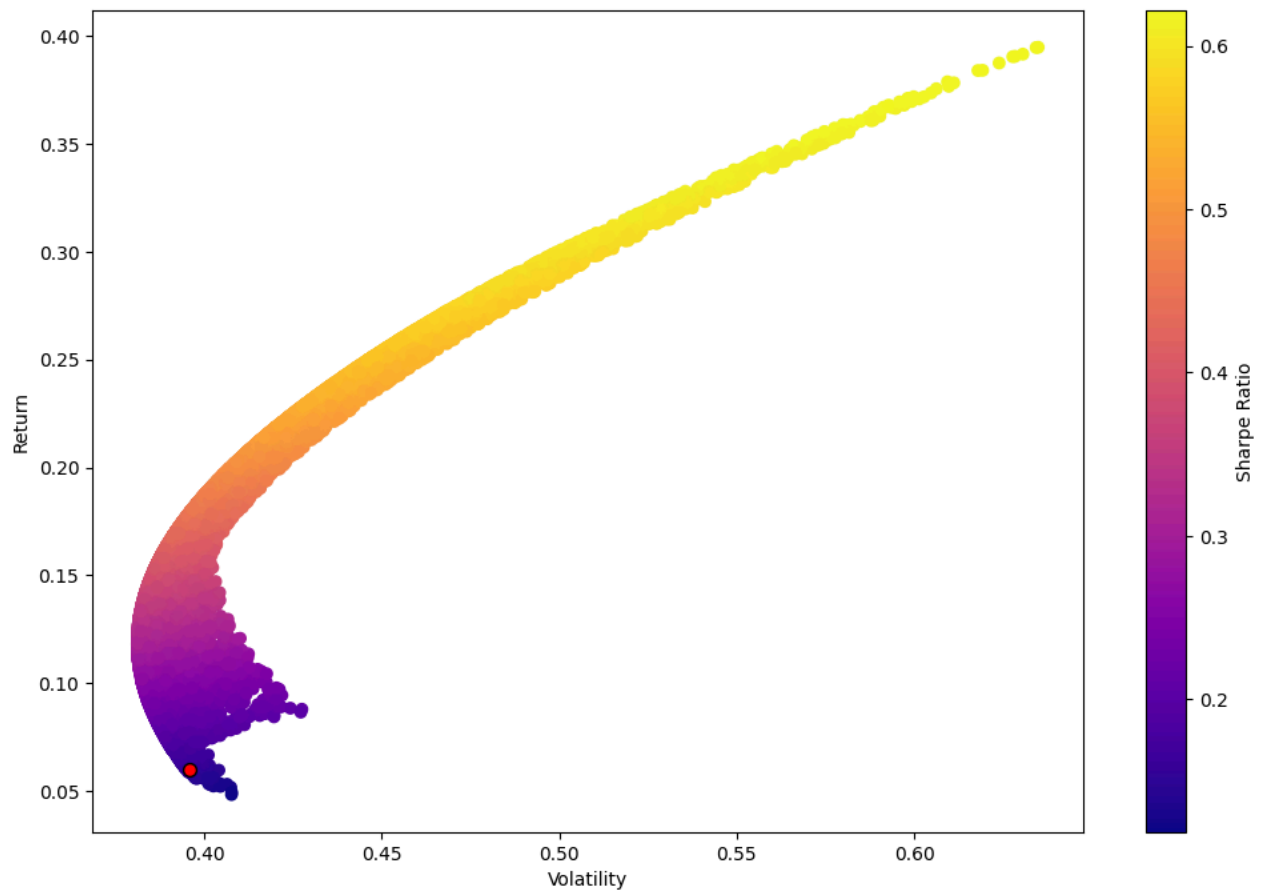
Plotting the data

```

plt.figure(figsize=(12,8))
plt.scatter(vol_arr,ret_arr,c=sharpe_arr,cmap='plasma')
plt.colorbar(label='Sharpe Ratio')
plt.xlabel('Volatility')
plt.ylabel('Return')

plt.scatter(max_sr_vol,max_sr_ret,c='red',s=50,edgecolors='black')
plt.show()

```



✦ Mathematical Optimization

✦ Functionalize Return and SR operations

```
def get_ret_vol_sr(weights):
    """
    Takes in weights, returns array of return, volatility, sharpe ratio
    """
    weights = np.array(weights)
    ret = np.sum(log_ret.mean() * weights) * 252
    vol = np.sqrt(np.dot(weights.T, np.dot(log_ret.cov() * 252, weights)))
    sr = ret/vol
    return np.array([ret, vol, sr])
```

```
from scipy.optimize import minimize
```

Optimization works as a minimization function, since we actually want to maximize the Sharpe Ratio, we will need to turn it negative so we can minimize the negative sharpe (same as maximizing the positive sharpe)

```
def neg_sharpe(weights):
    return get_ret_vol_sr(weights)[2] * -1
```

```
def check_sum(weights):
    """
    Returns 0 if sum of weights is 1.0
    """
    return np.sum(weights) - 1
```

```
cons = ({'type': 'eq', 'fun': check_sum})
```

```
# 0-1 bounds for each weight
bounds = ((0, 1), (0, 1), (0, 1))
```

```
# Initial Guess
init_guess = [0.33, 0.33, 0.33]
```

```
# Sequential Least Squares Programming (SLSQP).
opt_results = minimize(neg_sharpe,init_guess,method='SLSQP',bounds=bounds,constraints=cons)
```

opt_results

```
message: Optimization terminated successfully
success: True
status: 0
  fun: -0.6222334691057593
   x: [ 1.000e+00  1.180e-16  1.041e-17]
  nit: 6
 jac: [-0.000e+00  2.029e-03  7.010e-02]
 nfev: 24
 njev: 6
```

opt_results.x

```
array([1.00000000e+00, 1.17961196e-16, 1.04083409e-17])
```

```
opt_opt_val = opt_results.fun
```